

STRATEGIC VOCABULARY

Domain = the whole problem space ("sell products online, checkout→fulfillment").

- **Core** subdomain — the differentiator, most design effort (order placement & fulfillment orchestration).
- **Supporting** subdomain — needed, custom, not a differentiator (Inventory, Shipping).
- **Generic** subdomain — buy/reuse, not modeled (Customer — only `CustomerId` referenced, no `Customer` aggregate).

Ubiquitous language — one vocabulary shared by model, code and domain experts, consistent only *inside* one bounded context.

Bounded context — the solution-space boundary where a term has exactly one meaning, one type, one implementation — and, in a microservice, the natural **database-per-service** boundary too (Order/Inventory/Payment/Shipping each own their schema).

Decomposition: **by subdomain** (event storming) or **by business capability** — the two lenses usually converge.

CONTEXT-MAP RELATIONSHIPS

Pattern	One-liner
Partnership	Succeed/fail together; no formal contract, plan as equals.
Shared kernel	Small, explicitly agreed model subset; changed only jointly.
Customer-supplier	Downstream depends on upstream; supplier plans around it.
Conformist	Downstream just conforms to upstream, no translation.
Anticorruption layer	Downstream isolates itself behind a translation layer.
Open host service / published language	Stable protocol + neutral schema any consumer can integrate against.
Separate ways	No meaningful integration need — deliberately don't integrate.
Big ball of mud	Diagnosis, not a pattern: boundaries undrawn, models bleed together.

TACTICAL BUILDING BLOCKS

Concept	Definition	Eq.	M	In
			ut	.
Entity	Identity persists across change (<code>Order</code>)	id	yes	domain
Value obj.	No identity (<code>Money</code> , <code>Address</code>)	attribute	no	domain
Aggregate	Cluster+root, one consistency boundary (<code>Order</code>)	root	no	domain
Repository	In-memory-collection illusion, one per root	—	—	domain / infra
Factory	Builds valid instances, enforces creation invariants (<code>OrderPlace</code>)	—	—	domain
Domain svc.	Cross-entity logic, stateless (<code>PricingService</code>)	—	—	domain
App. svc.	Use-case orchestrator, stateless: load→invoke→save→publish (<code>PlaceOrderService</code>)	—	—	application
Domain event	Past-tense fact, already happened (<code>OrderPlaced</code>)	event	no	domain

Arrows: application → repository/root/domain-service; the domain layer never depends back on application; only the domain-owned repository interface is depended on, never its impl.

CODE-LAYOUT SKELETON

```
com.example.order
├── domain
│   ├── event
│   ├── application
│   ├── infrastructure
│   │   └── acl
│   ├── api
│   └── saga
com.example.shared
• domain — Order, OrderLine, OrderStatus, OrderId, OrderRepository; event: OrderPlaced / Approved / Rejected / Cancelled.
• application — PlaceOrderService, PlaceOrderCommand.
• infrastructure — JPA mapping, repo impl, outbox/publisher adapters; acl: anticorruption translators.
• api — REST controllers, saga command handlers, event consumers.
• saga — PlaceOrderSaga state machine + saga commands/replies.
• com.example.shared — CustomerId, ProductId, Money, Address, DomainEvent, DomainEventEnvelope.
domain → nothing.
application → domain only.
infrastructure / api / saga → domain+application, never back.
```

AGGREGATE RULES

1. Only the **root** is referenced from outside (`OrderLine` never handed out alone).
2. Other aggregates referenced by **identity** only — `CustomerId`, `ProductId`, never an object reference.
3. **One transaction** creates/updates exactly **one aggregate**.
4. Protect true **invariants** inside the boundary, and nothing else.
5. Keep aggregates **small**.

Smaller/separate aggregates (e.g. standalone `Order`, not nested in `Customer`) → less lock contention, bounded tx size, cleaner context fit; cost: cross-aggregate coordination moves to events/sagas.

WHY NOT DISTRIBUTED TX

- **2PC/XA unsupported** — brokers (Kafka, RabbitMQ), NoSQL stores, REST are not XA resource managers.
- **Availability = product of participants'** $0.995^4 \approx 98.0\%$ for the 4-step Place Order Saga — far worse than any one participant alone.
- **Locks held network-wide** for the whole prepare/commit window — one slow participant stalls every other's throughput.
- **CAP:** a cloud built to restart/relocate/retry looks like a partition to a 2PC coordinator, which must block, not guess.

Consequence: **one transaction, one aggregate, one service** — consistency between services is eventual, via sagas & events.

SPRING MAPPING

Concept	Spring
Aggregate root	<code>@Entity + @Version</code>
Value object	<code>record + @Embeddable / @ElementCollection</code>
Repository (port)	plain interface, domain layer
Repository (impl)	<code>JpaRepository</code> - backed adapter
Application service	<code>@Service + @Transactional</code>
Event, in-process	<code>@TransactionalEventListener / @ApplicationModuleListener</code>
Outbox	Modulith event publication registry / <code>@Externalized</code>
Saga command handler	<code>@KafkaListener</code>
Projector (CQRS)	<code>@KafkaListener + upsert</code>
Module boundary	<code>@ApplicationModule (Modulith)</code>

SAGAS

A **sequence of local transactions** with compensations — never one distributed transaction: `T1...Tn, Cn...C1`.

- **Compensatable** — reserve inventory → `ReleaseInventoryCommand` undoes it.
- **Pivot** — authorize payment; point of no return, no compensation modeled.
- **Retriable** — schedule shipping; retried with backoff, must be idempotent.

Choreography — no coordinator, participants react to each other's events (loose, but no single place reads the flow). **Orchestration** — `PlaceOrderSaga` sends explicit commands, waits for replies (recommended for all but the simplest sagas).

Sagas are **ACD, not ACID** — no isolation: lost update, dirty read, fuzzy read are all possible.

Six countermeasures:

- **Semantic lock** — explicit pending flag (`APPROVAL_PENDING`).
- **Commutative updates** — order of ops stops mattering.
- **Pessimistic view** — run the riskier step first.
- **Reread value** — recheck `@Version` before overwrite.
- **Version file** — append-only record of ops (the outbox row).
- **By value** — pass data, not just an id, to avoid a re-read race.

DOMAIN-EVENT CHECKLIST

- **Past tense** naming — `OrderPlaced`, never `PlaceOrder`.
- **Minimal** (id + fact, e.g. `OrderApproved`) vs **enriched** (carries what known consumers need, e.g. `OrderPlaced`'s lines/total) — enrichment costs contract stability.
- **Envelope:** `eventId` (dedup), `aggregateType / aggregateId` (routing), `causationId` (direct cause), `correlationId` (whole saga).
- **Idempotent consumer** — natural upsert, or a `processed_events` table keyed on `eventId`, written in the same local tx as the side effect.
- **Ordering** — partition/key by `aggregateId` so same-aggregate events stay in order.
- **Versioned, never mutated** — a new fact needs a new event type.

OUTBOX · LISTEN-TO-YOURSELF · EVENT SOURCING

	Outbox	Listen-to-self	Event sourcing
Writes published	Aggregate+outbox row in one local tx; relay publishes after.	Publish first; local write happens after, driven by consuming it.	The event stream itself is the write.
Extra infra	Outbox table + relay (poller/CDC).	None — broker topic only.	EventStore (append-only stream).
Source of truth	The aggregate's row.	The row is transient.	The event stream, folded on load.

CQRS

Command side = `Order + PlaceOrderService` + published events (write-optimized, normalized).

Query side = read model(s): purpose-built, denormalized, fed by the same events (`OrderHistoryView / Projector / QueryService`).

Any number of independent read models can subscribe to one stream; the command side never knows how many. Trade-off: eventual/stale reads vs. query scalability and shape freedom.

EVENT SOURCING

`EventStore.append(streamId, expectedVersion, events) / load(streamId)` — persist the aggregate as its append-only stream, not one overwritten row. Current state = `fold(apply, events)`, replayed from scratch on every load.

- **decide(cmd)** — validates, returns events; never mutates.
- **apply(event)** — pure mechanical state change; never fails.

Free: full history, audit log, reliable publication. Cost: snapshots at scale; evolve via a new event type/version, never repurpose a field.